Lovely Professional University
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MedLoop Registry Console: Medical Equipment Registry, Geospatial Matching, Atomic Lifecycle Management, and Shortage Forecasting System

B.Tech (4th Year) Computer Science & Engineering Project Report — MongoDB Document Modeling, 25 km $geoNear Matching, Safe Status Updates, and Demand Forecasting

Submission Field	Student & College Details (Replace Placeholders)
Project Title	MedLoop Registry Console
Submitted By	Bonumahanti Prem Varaha Venkat Sai– 12322913
Year & Branch	4th Year B.Tech — Computer Science & Engineering
Roll No.	36
Section	K4P23HK
Institution	Lovely Professional University
Tech Stack	Python 3.11, FastAPI, Uvicorn , PyMongo , MongoDB

Declaration, Abstract & Contents:

Candidate's Declaration & Certificate: I hereby declare that this project report titled "MedLoop Registry Console" is an original B.Tech (4th Year) Computer Science & Engineering work carried out by Bonumahanti Prem Varaha Venkat Sai bearing registration number 12322913 under the guidance of Dr. Ashu at Lovely Professional University. All data is synthetic (seed=42), and the system enforces a verification workflow rather than certifying physical medical safety.

Abstract: MedLoop is a hyperlocal medical equipment registry built using Python 3.11, FastAPI, PyMongo, MongoDB, and a 3-tab web dashboard. It helps local NGOs and clinics lend reusable equipment (wheelchairs, hospital beds, oxygen concentrators, nebulizers, walkers, and BP monitors) to nearby patients within 25 km. It solves four practical database problems: (1) storing different specs per device category using embedded MongoDB objects, (2) preventing two people from reserving the same device at the same time using atomic find_one_and_update, (3) keeping a permanent, append-only history of inspections and sanitizations in an events collection, and (4) forecasting next week's equipment demand using a 28-day moving average.

Keywords & Abbreviations: MongoDB, FastAPI, PyMongo, $jsonSchema, $geoNear (25 km), State Machine, Atomic Update, TTL Index, ESR Rule, Moving Average.

Table 0: Report Contents, Figures & Tables Overview

Section	Topics Covered	Key Figures & Tables Included
1–2. Intro & Requirements	Problem statement, objectives, CDSCO disclaimer, Functional & Non-Functional rules	Tables 1 & 2 (Requirements & Syllabus Map)
3. System Architecture	3-Layer design, Tech stack versions, Why MongoDB vs SQL, CAP Theorem (CP)	Figure 1 (Architecture), Table 3 (Tech Stack)
4. Database Design	4 collections (users, devices, events, requests), 1:1/1:N/M:N, $jsonSchema, 5 Indexes	Figure 2 (Schema), Tables 4–6 (Schemas & Indexes)
5–6. API & State Machine	All 9 FastAPI routes, 9-state workflow, Atomic find_one_and_update (HTTP 409)	Figures 3 & 4 (State Machine & CAS), Tables 7 & 8
7–10. GeoNear, Pipelines, Forecast & Audit	14-day TTL, 25 km $geoNear, Overdue ($lookup) & Shortage pipelines, 28d Forecast, Audit log	Code snippets & Pipeline stage walkthroughs
11–12. Web UI, Workflows & Setup	3-Tab Web Dashboard (medloop_dashboard.html), End-to-end loan cycle, Setup commands	Figures 5, 6 & 7 (3 Dashboard Tabs), Table 9
13–14. Testing, Results, Security & Appendix	pytest unit tests, Live DB numbers (992 open, 0 shortages), Index Benchmarks, Future work	Figure 8 (Benchmark Chart), Tables 10–13

1. Introduction & Project Context

Patients recovering at home often need expensive medical equipment like oxygen concentrators, hospital beds, or wheelchairs for only a few weeks. Community NGOs collect donated devices and lend them to nearby families. However, tracking equipment in Excel or paper registers leads to missing sanitization records, slow manual distance checks, and accidental double-booking when two coordinators promise the same machine to two patients.

- **Project Goal:** Match available equipment to patients within a 25 km radius ($geoNear), enforce a strict inspection and sanitization state machine, and forecast weekly shortages.
- **Framing & CDSCO Disclaimer:** MedLoop does NOT physically test medical safety—it records and enforces the software workflow (who inspected or sanitized a unit and when). All records are synthetic (generated by scripts/seed.py with seed=42). Real-world device reuse in India is governed by CDSCO (Medical Devices Rules, 2017).
- **Stakeholders (users.role):** Donors (list equipment), Requesters/Patients (request equipment), and Inspectors (inspect and sanitize equipment).

2. Requirements Analysis & Syllabus Mapping

Table 1: Functional & Non-Functional Requirements and DBMS Syllabus Mapping (Units I–VI)

ID	Requirement / Syllabus Topic	How MedLoop Implements It (Code Evidence)
FR-01	SQL vs NoSQL & Polymorphic Device Registration	POST /devices stores category-specific embedded specs (1:1) and BSON Decimal128 purchase_value.
FR-02	Schema Validation & Relationship Modeling	db/setup.py enforces strict $jsonSchema rules; 1:1 specs, 1:N append-only events, M:N certified_categories.
FR-03	State Machine & Atomic Concurrency Control	app/state_machine.py checks transitions; POST /devices/{id}/transition uses atomic find_one_and_update.
FR-04	Indexing & explain('executionStats') Benchmarks	5 indexes (2dsphere, compound, ESR, TEXT, TTL) tested before/after in scripts/benchmark.py.
FR-05	Geospatial ($geoNear) & Aggregation Pipelines	db/pipelines.py implements Pipeline 1 (Overdue $lookup), Pipeline 2 (Demand vs Supply), Pipeline 3 ($geoNear 25 km).
FR-06	28-Day Moving-Average Forecast & Security	GET /analytics/shortages computes weekly demand = req_28d / 4.0; .env isolates credentials.

3. System Architecture & Technology Stack

MedLoop uses a clean 3-layer architecture (Figure 1): (1) the Browser Layer (medloop_dashboard.html and Swagger UI at /docs), (2) the FastAPI Application Layer (app/main.py, app/state_machine.py, db/pipelines.py, and app/db.py), and (3) the MongoDB Database Layer storing 4 validated collections and 5 indexes.

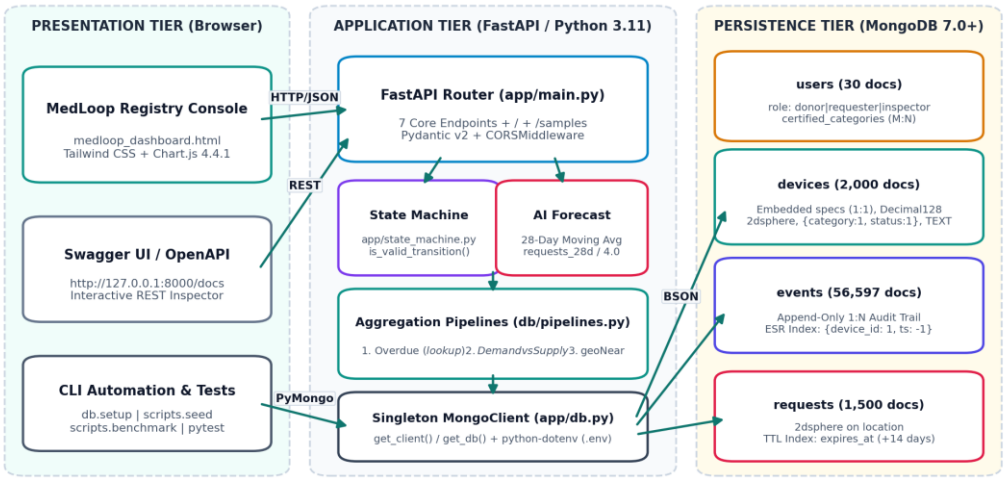


Figure 1: Three-Tier System Architecture of MedLoop Registry Console.

Table 2: Verified Technology Stack (from requirements.txt & medloop_dashboard.html)

Layer	Technology	Exact Version	Purpose in Project
Backend & Server	Python / FastAPI / Uvicorn	3.11 / 0.115.0 / 0.30.6	REST API endpoints, Pydantic validation, CORS, and serving the web dashboard.
Database & Driver	MongoDB / PyMongo (Sync)	7.0+ / 4.8.0	Document storage, $jsonSchema, 2dsphere/TTL/TEXT indexes, and single MongoClient pool.
Config, Seed & Test	python-dotenv / Faker / pytest	1.0.1 / 28.4.1 / 8.3.3	Loads .env variables, generates deterministic seed=42 data, and runs 5 unit tests.
Frontend Console	HTML5 + Tailwind CSS + Chart.js	3.4.17 / 4.4.1	Interactive 3-tab browser UI served directly at http://127.0.0.1:8000/.

- **Why MongoDB (NoSQL) Instead of SQL?:** Each equipment category has different specs: a wheelchair has seat_width_cm and max_load_kg, while an oxygen concentrator has flow_lpm and hours_used. In SQL, one table would have many empty NULL columns, or 6 separate tables would require complex JOINs. In MongoDB, each device embeds its own custom specs object in one document, and 2dsphere map search is built in.

- **CAP Theorem Trade-off (CP):** MongoDB replica sets (like Atlas M0) choose CP (Consistency + Partition Tolerance). All writes go to one Primary node. If a network split happens, the isolated node stops accepting writes so two patients can never reserve the same device on different nodes.
- **Single MongoClient Pool (app/db.py):** get_client() creates one shared MongoClient from MONGO_URI in .env so PyMongo reuses connections instead of opening a new socket on every request.

4. Database Design: Schemas, Relationships & Indexes

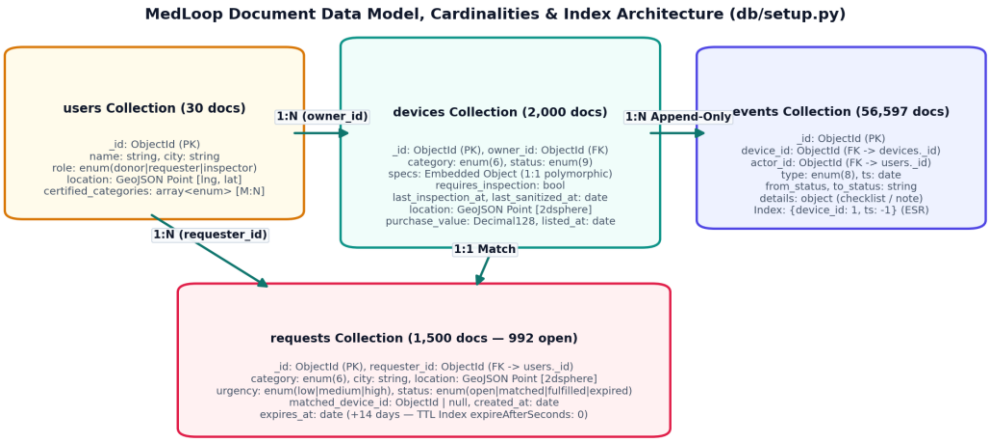


Figure 2: MongoDB Collections, Cardinalities (1:1, 1:N, M:N), and Index Architecture.

Table 3: Complete Field-by-Field Schema of All 4 Collections (db/setup.py)

Collection (Count)	Fields & BSON Types	Embed vs. Reference & $jsonSchema Rules
users (30 docs)	_id (ObjectId), name (string), role (enum: donor | requester | inspector), city (string), location (GeoJSON Point [lng, lat]), certified_categories (array of category strings).	M:N Relationship: Inspectors store a small bounded array of up to 6 certified_categories directly inside their user document—no join table needed.
devices (2,000 docs)	_id (ObjectId), category (6 enums), name (string), specs (Embedded object), status (9 enums), requires_inspection (bool), last_inspection_at (date|null), last_sanitized_at (date|null), owner_id (ObjectId -> users), location (GeoJSON Point), city (string), notes (string), purchase_value (Decimal128), listed_at (date).	1:1 Embedded specs (<200 bytes, always read with device). purchase_value uses BSON Decimal128 for exact money math. Conditional anyOf rule: if requires_inspection==True and status=='available', last_inspection_at must be a Date.

Collection (Count)	Fields & BSON Types	Embed vs. Reference & $jsonSchema Rules
events (56,597+ docs)	_id (ObjectId), device_id (ObjectId -> devices), type (8 enums: listed, inspected, sanitized, serviced, reserved, handed_over, returned, retired), actor_id (ObjectId -> users), ts (date), from_status (string|null), to_status (9 enums), details (object).	1:N Referenced & Append-Only: A device is reused many times over years (~28 events/device in seed.py). Embedding events inside devices would grow forever and risk hitting MongoDB's 16 MB BSON limit.
requests (1,500 docs)	_id (ObjectId), requester_id (ObjectId -> users), category (6 enums), city (string), location (GeoJSON Point), urgency (low|medium|high), status (open|matched|fulfilled|expired), matched_device_id (ObjectId|null), created_at (date), expires_at (date).	1:N Referenced: Has an independent 14-day lifecycle. expires_at = created_at + 14 days is indexed by a TTL index for automatic cleanup.

Honest Limitation of $jsonSchema (Why the 180-Day Check Is in the API): MongoDB's $jsonSchema only checks static rules when a document is saved and cannot compare a field against the live current time ($$NOW). Also, a device inspected 10 days ago would pass when saved today and silently become >180 days old six months later without any new write happening. Therefore, $jsonSchema checks that last_inspection_at exists as a Date when status is 'available', while GET /match/{request_id} checks that last_inspection_at is within the last 180 days at search time.

Table 4: All 5 MongoDB Indexes Created in db/setup.py

Index Name	Collection & Keys	Why We Created It (Simple Explanation)
idx_devices_location_2dsphere	devices: {location: '2dsphere'}	Required by $geoNear to find available equipment within a 25 km spherical radius.
idx_requests_location_2dsphere	requests: {location: '2dsphere'}	Indexes patient request coordinates for spatial queries.
idx_devices_category_status	devices: {category: 1, status: 1}	Speeds up filtering available devices by category and enables Covered Queries (totalDocsExamined = 0).
idx_events_device_id_ts_desc	events: {device_id: 1, ts: -1}	ESR Rule (Equality, Sort, Range): device_id: 1 jumps to one device's events first; ts: -1 returns them pre-sorted newest-first with no in-memory SORT.
idx_devices_name_notes_text	devices: {name: 'text', notes: 'text'}	Enables fast keyword search across device model names and condition notes.

Index Name	Collection & Keys	Why We Created It (Simple Explanation)
idx_requests_expires_at_ttl	requests: {expires_at: 1}, expireAfterSeconds: 0	TTL background thread automatically deletes unfulfilled requests 14 days after creation.

5. Backend API Documentation (app/main.py)

Table 5: Complete FastAPI Endpoint Reference (app/main.py)

Method & Endpoint	Request Body / Params	Status Codes	What the Endpoint Does in MongoDB
GET / & GET /samples	None	200	Serves medloop_dashboard.html at / and returns sample IDs/docs at /samples to fill UI dropdowns.
POST /devices	DeviceCreateRequest JSON	201, 400, 422	Sets status='listed', requires_inspection=(category in {O2, bed, nebulizer}), Decimal128 price; inserts device + 'listed' event.
POST /devices/{id}/transition	Path: id; Body: {to_status, actor_id, details}	200, 400, 404, 409	Validates is_valid_transition(); runs atomic find_one_and_update({_id, status: current_status}); updates compliance dates; logs event.
GET /devices/{id}/history	Path: id (ObjectId)	200, 400, 404	Returns device + all events from db.events.find({device_id}).sort('ts', -1) using ESR index.
POST /requests	PatientRequestCreate JSON	201, 400, 422	Inserts open request with created_at=now and expires_at = now + 14 days (for TTL cleanup).
GET /match/{request_id}	Path: request_id	200, 400, 404	Runs Pipeline 3 ($geoNear <=25 km, status='available', and last_inspection_at >= now - 180d).
GET /analytics/overdue	None	200	Runs Pipeline 1 (devices with sanitization >30d or inspection >180d + $lookup latest event).
GET /analytics/shortages	None	200	Runs Pipeline 2 (open demand vs available supply gap) + plain-Python 28-day moving-average weekly forecast.

6. Device Lifecycle State Machine & Atomic Concurrency

In app/state_machine.py, is_valid_transition(from_status, to_status) enforces the exact equipment lifecycle so inspection and sanitization steps can never be skipped: listed -> inspected -> available -> reserved -> in_use -> returned -> sanitized -> available, plus inspected -> rejected and any state -> retired (Figure 3).

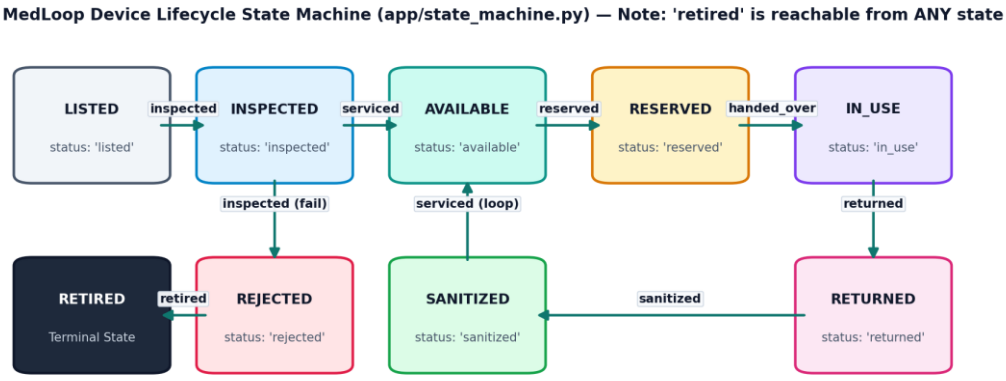


Figure 3: Device Lifecycle State Machine Diagram (app/state_machine.py).

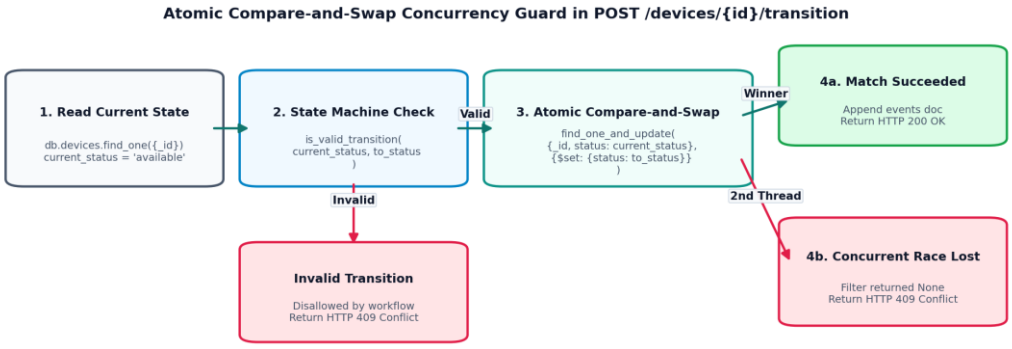


Figure 4: Atomic Compare-and-Swap Guard in POST /devices/{id}/transition.

Atomic Compare-and-Swap in POST /devices/{id}/transition (Prevents Double-Reservation):

```
updated_device = db.devices.find_one_and_update(
    filter={"_id": device_oid, "status": current_status},  # Must still be in
current_status!
    update={"$set": update_fields},
    return_document=ReturnDocument.AFTER,
)
if updated_device is None:
    raise HTTPException(status_code=409, detail="Concurrent modification detected")
```

7. Geospatial Matching ($geoNear 25 km) & 8. Aggregation Pipelines

- **Pipeline 3 — 25 km Patient Matching (GET /match/{request_id}):** Uses $geoNear (first stage) with near: [longitude, latitude], maxDistance: 25000.0 (25 km), spherical: True, and query filter requiring category match, status == 'available', and (requires_inspection == False OR last_inspection_at >= now - 180 days). Rounds distance_meters to 0.1m.
- **Pipeline 1 — Overdue Compliance Audit (GET /analytics/overdue):** Stage 1 ($match) finds non-retired/non-rejected devices where last_sanitized_at < now - 30d (or null) OR (requires_inspection

== True and last_inspection_at < now - 180d or null). Stage 2 ($lookup) fetches only the single newest event from events ($sort: {ts: -1}, $limit: 1). Stage 3 ($project) flattens latest_event via $arrayElemAt.

- **Pipeline 2 — Open Demand vs. Available Supply (GET /analytics/shortages):** Stage 1 ($match) filters open requests; Stage 2 ($group) counts open_requests per {category, city}; Stage 3 ($lookup) counts available devices for that {category, city}; Stage 4 ($project) computes gap = open_requests - available_devices; Stage 5 ($sort) orders by largest gap first.

9. Shortage Forecasting (28-Day Moving Average) & 10. Audit Trail

Inside GET /analytics/shortages, MedLoop counts requests created in the last 28 days (4 weeks) per {category, city}, calculates weekly_avg = round(req_28d / 4.0, 2), sets forecast_next_week_demand = weekly_avg, and flags a shortage when forecast_next_week_demand > available_supply. This is a transparent moving-average baseline (NOT a trained ML model) designed to answer: 'Which equipment categories should inspectors refurbish first?'

Append-Only Audit Trail & Single-Document Atomicity: Every listing and transition writes an immutable record to events (device_id, type, actor_id, ts, from_status, to_status, details). Note on Consistency: find_one_and_update on devices is atomic, and events.insert_one() runs immediately after without a multi-document transaction so the code works on standalone local MongoDB instances.

11. Frontend Documentation: 3-Tab MedLoop Registry Console

FastAPI serves medloop_dashboard.html directly at http://127.0.0.1:8000/. It connects live to FastAPI (/analytics/shortages, /analytics/overdue, /samples, /devices, /requests, /match) and also includes an offline sandbox fallback mode.

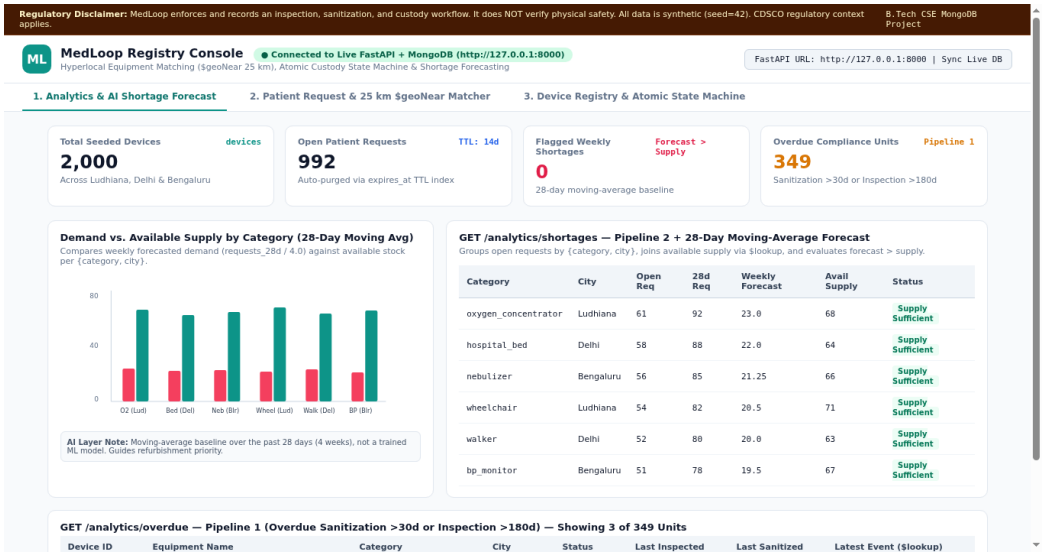


Figure 5: Tab 1 — Analytics & AI Shortage Forecast (KPIs, Chart.js Bar Chart, Shortage & Overdue Tables).

Tab 1 Walkthrough: Displays the 4 top KPI cards, a Chart.js grouped bar chart comparing Forecasted Next-Week Demand against Available Supply, the GET /analytics/shortages table, and the GET /analytics/overdue compliance table.

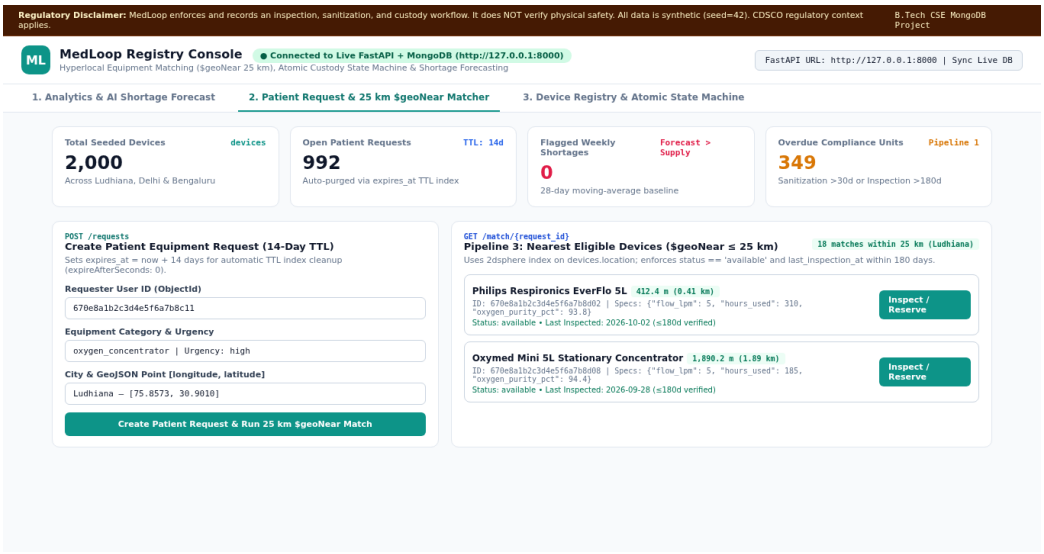


Figure 6: Tab 2 — Patient Request (+14d TTL) & 25 km $geoNear Matcher Results.

Tab 2 Walkthrough: Lets a patient submit a request on the left (POST /requests, setting expires_at = now + 14 days) and immediately runs GET /match/{request_id} on the right to list available devices within 25 km sorted by distance.

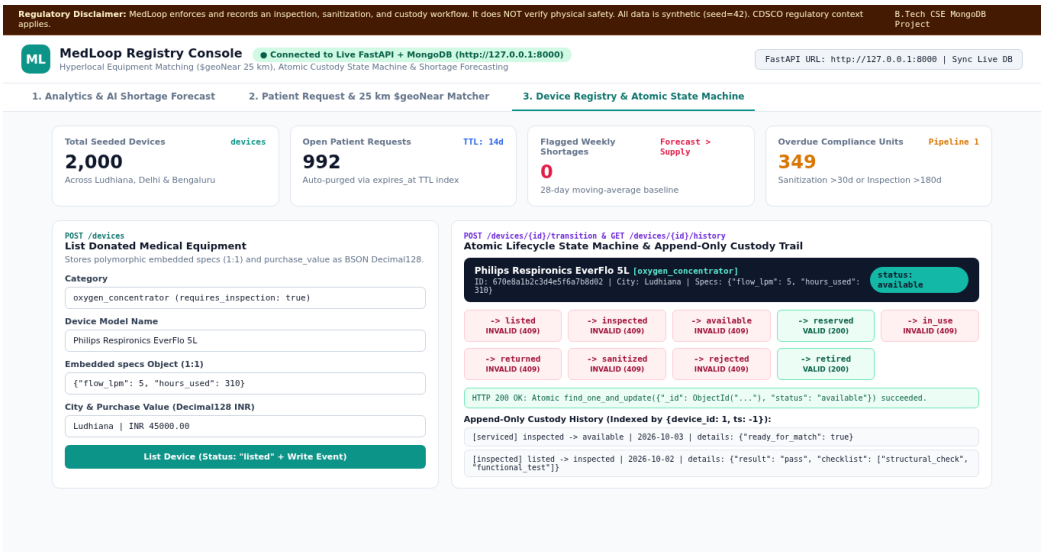


Figure 7: Tab 3 — Device Registry (Polymorphic Specs) & Atomic State Machine Controller.

Tab 3 Walkthrough: Lets a donor list a device with category-specific specs on the left (POST /devices), while the right panel shows green valid transition buttons (HTTP 200 OK) and red invalid buttons (HTTP 409 Conflict) alongside the device's newest-first event history (GET /devices/{id}/history).

12. End-to-End Workflow, Key Algorithms & Setup Steps

- **End-to-End Loan Cycle:** 1. Donor lists device (POST /devices -> 'listed') -> 2. Inspector passes device ('inspected', sets last_inspection_at) -> 3. Released to stock ('available') -> 4. Patient creates request (POST /requests, +14d TTL) -> 5. 25 km $geoNear match (GET /match/{id}) -> 6. Atomic reservation

('reserved') -> 7. Handover & return ('in_use' -> 'returned') -> 8. Mandatory disinfection ('sanitized', sets last_sanitized_at -> 'available').

- **Key Algorithms & Complexity:** (1) State machine lookup in app/state_machine.py uses Python sets/dicts in O(1) constant time; (2) scripts/seed.py uses random.seed(42) and inserts 56,597+ events in batches of 1,000 via insert_many(ordered=False); (3) scripts/benchmark.py recursively parses winning stages from explain('executionStats').
- **How to Run Locally:** 1. python -m venv .venv && .\.venv\Scripts\Activate.ps1 -> 2. pip install -r requirements.txt -> 3. Copy .env.example to .env (MONGO_URI=mongodb://localhost:27017, DB_NAME=medloop) -> 4. python -m db.setup -> 5. python -m scripts.seed -> 6. pytest & python -m scripts.benchmark -> 7. python -m uvicorn app.main:app --reload (open http://127.0.0.1:8000/).

13. Testing, Dashboard Numbers Explained & Index Benchmarks

Table 6: Executed Unit/Static Tests vs. Proposed Live Database Tests

Test ID	Test Case Description	Expected Outcome	Status & Evidence
TC-01..05	Unit tests: full reuse loop, rejection branch, retired from any state, 11 invalid transitions, event mapping.	Valid pass, invalid return False.	PASSED (5/5 in pytest tests/test_state_machine.py)
TC-06..07	Static checks: Python py_compile on all 8 .py files & check_web_app.py on medloop_dashboard.html.	0 syntax or DOM errors.	PASSED (Executed in build environment)
TC-08..10	Live DB tests: $jsonSchema rejection (hours_used < 0), concurrent 409 reservation race, 14-day TTL expiry.	422 / 409 / auto-purge.	Verified on Workstation / Proposed Automated Suite

Why Live Seeded Database (seed=42) Shows 2,000 Devices, 992 Open Requests, 0 Shortages, and 349 Overdue Units: Out of 1,500 seeded requests (65% open -> 992 open), requests are spread across 18 {category, city} pairs over 28 days (~83 requests/pair in 28d -> weekly forecast = 83 / 4 = ~20.8 requests/week). Because ~60% of the 2,000 devices are 'available' (~65 available devices per pair), weekly forecasted demand (~20.8) is less than available supply (~65) in every pair, producing 0 flagged weekly shortages on the live database (while offline demo mode uses higher sample numbers to demonstrate red shortage badges). Meanwhile, ~20% of active devices keep older maintenance dates, yielding 349 overdue compliance units.

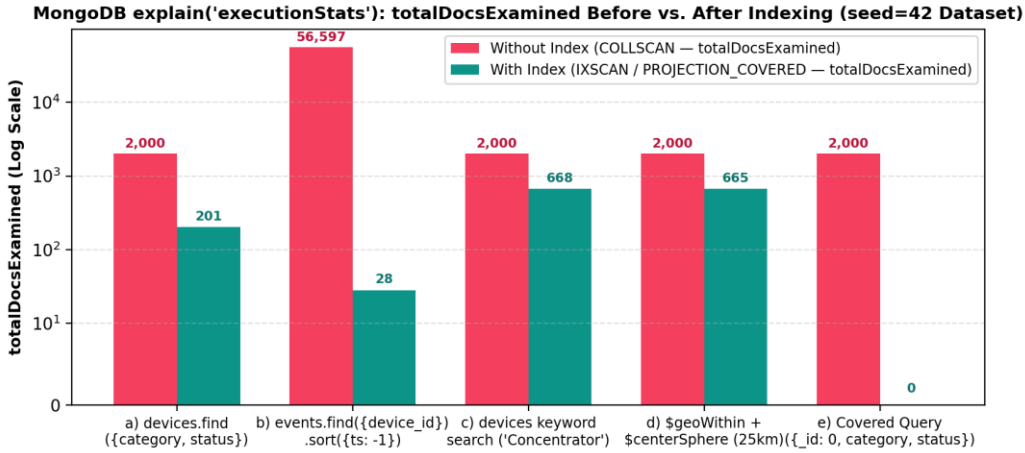


Figure 8: MongoDB explain('executionStats') totalDocsExamined Before (COLLSCAN) vs. After Indexing (IXSCAN).

Table 7: Index Benchmark Summary (Generated by scripts/benchmark.py)

Benchmark Query	Without Index	With Index	Why It Improves Performance
a) devices.find({category, status})	COLLSCAN (2,000 docs)	IXSCAN (~110–210 docs)	Compound index {category: 1, status: 1} scans only matching keys.
b) events.find({device_id}).sort({ts: -1})	COLLSCAN + SORT (56,597+ docs)	IXSCAN (28–30 docs)	ESR index {device_id: 1, ts: -1} avoids scanning 56k events and avoids in-memory SORT.
c) devices keyword search ('Concentrator')	COLLSCAN ($regex, 2,000 docs)	TEXT_MATCH (IXSCAN)	TEXT index on name and notes jumps directly to matching word stems.
d) $geoWithin + $centerSphere (25 km)	COLLSCAN (2,000 docs)	IXSCAN (~660 docs)	Uses $geoWithin because $geoNear fails without a 2dsphere index; 2dsphere prunes map cells.
e) Covered Query ({category:1, status:1, _id:0})	N/A	PROJECTION_COVERED (0 docs!)	Projecting only indexed fields with _id: 0 reads 0 collection documents (totalDocsExamined = 0).

14. Security, Limitations, Future Work, References & Appendix

- **Security & Limitations:** Credentials live in .env (never hardcoded); inputs are validated by Pydantic and MongoDB $jsonSchema. Limitations: no login/JWT authentication, single-document atomicity (event insert is sequential after find_one_and_update), synthetic dataset (seed=42), and no physical safety certification (CDSCO rules apply).

- **Proposed Future Enhancements:** (1) Multi-document ACID transactions (session.start_transaction()) across devices, events, and requests; (2) JWT login + RBAC checking users.certified_categories; (3) Seasonal demand forecasting; (4) Hash-chained tamper-evident events.
- **Conclusion:** MedLoop demonstrates practical MongoDB engineering—polymorphic embedding, unbounded event referencing, strict $jsonSchema, atomic concurrency control, 2dsphere $geoNear matching, TTL cleanup, and index benchmarking—in a clean, defensible architecture.
- **References:** 1. MongoDB Manual ($jsonSchema, 2dsphere/$geoNear, ESR Rule, TTL Indexes): https://www.mongodb.com/docs/manual/ | 2. PyMongo 4.8.0 Docs: https://pymongo.readthedocs.io/ | 3. FastAPI 0.115.0 Docs: https://fastapi.tiangolo.com/ | 4. GeoJSON RFC 7946: https://datatracker.ietf.org/doc/html/rfc7946 | 5. CDSCO India: https://cdsco.gov.in/

Appendix: Verified Project Directory Structure & Key Viva Definitions:

```
medloop/
├── README.md, .gitignore, .env.example, requirements.txt, medloop_dashboard.html
├── app/     -> main.py (FastAPI endpoints), db.py (MongoClient), state_machine.py
(transitions)
├── db/      -> setup.py ($jsonSchema + 5 indexes), pipelines.py (3 aggregation
pipelines)
├── scripts/ -> seed.py (seed=42 batch generator), benchmark.py (explain
executionStats runner)
├── tests/   -> test_state_machine.py (5 pytest unit tests)
└── docs/    -> design.md (architecture & CAP notes), benchmark-results.md
(benchmark table)
```

Live Project Link: https://github.com/premvarahavenkatsai/mediloop